Kernix benchmark report: baseline-0.1.0

Measured 2026-10-19T14:20:36Z on 12th Gen Intel(R) Core(TM) i3-1215U.

Rust: rustc 1.99.0 (b940084d7 2026-09-28); NumPy: 2.5.4; one CPU thread; float64.

Source: `b208e4231d00635eb4abb5c1647222f5779c43a9`; working tree dirty: True.

Comparative CPU results

Workload	Kernix p50 / p95 (us)	ndarray p50 / p95 (us)	NumPy p50 / p95 (us)
add_65536_f64	23.402 / 31.312	21.199 / 21.410	23.481 / 24.788
matmul_128_f64	3029.011 / 4304.708	83.317 / 88.888	705.774 / 910.168
matmul_32_f64	32.013 / 36.372	1.502 / 1.595	11.044 / 13.884
softmax_256x64_f64	397.788 / 424.374	67.230 / 69.091	94.602 / 108.892
sum_65536_f64	31.027 / 51.026	5.698 / 6.087	10.184 / 10.514

Method and limits

All operations allocate their output except scalar sums. Inputs are preallocated and contiguous. ndarray uses its default matrixmultiply engine (no BLAS feature); NumPy uses the build recorded in raw.json. BLAS/OMP thread counts are fixed to one. These are warmed, sequential wall-clock samples on a shared laptop; CPU frequency and other system load are uncontrolled. p95 is the maximum of 15 batch means, not a per-request tail-latency guarantee. Different sum orders may round differently. The ndarray softmax is an explicitly coded stable rowwise comparison, not a native ndarray softmax API. NumPy timings include Python call overhead. No GPU comparison. No claim that a complete Full HD model reaches 120 FPS follows from these measurements. raw.json contains all measurements, dependencies and environment metadata.